

UNIT 1 ACTIVITY 2

Top 5 Cities by Average Revenue per Category Using Hadoop MapReduce

Unit 1 - Assignment 1

Name : Ankna Litoriya

Roll no. : NITRB-25-BD-PG-003

Programme: M.Tech

Institute: NITTTR, Bhopal

Course: CSEB08 Big Data Tools

Activity Type: Practical + Analytical Activity

Course Outcome: CO1 – Use Big Data and its Business Implications

All github repos : <https://github.com/Ankna-litoriya/M.Tech>

1. Introduction

In modern e-commerce platforms, organizations operate across multiple cities and product categories. Every purchase transaction generates revenue based on the quantity of products purchased and the price per unit. Analyzing this revenue helps businesses understand which locations perform best and which product categories generate the most income.

Because large-scale e-commerce systems generate massive volumes of transaction data, traditional data processing techniques are not sufficient to analyze the data efficiently. Big Data technologies such as Hadoop MapReduce allow distributed processing of large datasets across clusters of computers.

This activity implements a multi-stage MapReduce solution to compute the Top 5 Cities by Average Revenue per Category from an e-commerce transaction dataset. The system processes transaction data stored in Hadoop Distributed File System (HDFS), calculates revenue for each transaction, computes the average revenue for each category-city pair, and finally ranks cities within each category to identify the top performers.

The main objective is to demonstrate how distributed computation can be used to perform aggregation, grouping, and ranking operations efficiently on large datasets.

2. Problem Statement

Given an e-commerce transaction dataset containing fields such as:

- Category
- City
- Quantity
- Price

Develop a Hadoop MapReduce program to:

1. Calculate the **average revenue for each (Category, City) pair**.
2. Identify the **Top 5 cities with the highest average revenue for each category**.

Revenue is calculated as:

Revenue = Quantity × Price

3. Sample Dataset

File Name: **ecommerce_transaction.csv**

```
TransactionID,UserID,ProductID,Category,City,Quantity,Price,PaymentMode,Date
T1001,U101,P501,Electronics,Mumbai,1,25000,CreditCard,2026-02-14
T1002,U102,P502,Fashion,Delhi,3,1500,UPI,2026-02-15
T1003,U103,P503,Grocery,Bangalore,5,200,Cash,2026-02-16
T1004,U104,P504,Electronics,Chennai,2,18000,DebitCard,2026-02-17
T1005,U105,P505,Fashion,Kolkata,4,2200,UPI,2026-02-18
T1006,U106,P506,Electronics,Hyderabad,1,32000,CreditCard,2026-02-18
```

This dataset represents transaction records from different product categories and cities.

```
hduser@Mtech: ~/ecommerce
hduser@Mtech:~$ whoami
hduser
hduser@Mtech:~$ start-dfs.ssh
start-dfs.ssh: command not found
hduser@Mtech:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Mtech]
2026-03-08 17:16:58,432 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
hduser@Mtech:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hduser@Mtech:~$ jps
3443 NameNode
3763 SecondaryNameNode
4022 ResourceManager
4150 NodeManager
4281 Jps
3567 DataNode
hduser@Mtech:~$ mkdir ecommerce
hduser@Mtech:~$ cd ecommerce
hduser@Mtech:~/ecommerce$ pwd
/home/hduser/ecommerce
hduser@Mtech:~/ecommerce$ nano ecommerce_transaction.csv
hduser@Mtech:~/ecommerce$ cat ecommerce_transaction.csv
TransactionID,UserID,ProductID,Category,City,Quantity,Price,PaymentMode,Date
T1001,U101,P501,Electronics,Mumbai,1,25000,CreditCard,2026-02-14
T1002,U102,P502,Fashion,Delhi,3,1500,UPI,2026-02-15
T1003,U103,P503,Grocery,Bangalore,5,200,Cash,2026-02-16
T1004,U104,P504,Electronics,Chennai,2,18000,DebitCard,2026-02-17
T1005,U105,P505,Fashion,Kolkata,4,2200,UPI,2026-02-18
T1006,U106,P506,Electronics,Hyderabad,1,32000,CreditCard,2026-02-18
hduser@Mtech:~/ecommerce$ hdfs dfs -mkdir /ecommerce
2026-03-08 17:20:33,409 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
hduser@Mtech:~/ecommerce$ hdfs dfs -put ecommerce_transaction.csv /ecommerce
2026-03-08 17:21:00,904 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
hduser@Mtech:~/ecommerce$ hdfs dfs -ls /ecommerce
2026-03-08 17:21:09,821 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

4. System Architecture

The solution uses **two MapReduce jobs**.

Job 1

Calculates **average revenue per (Category, City)**.

Steps:

- Mapper calculates transaction revenue.
- Reducer calculates average revenue.

Job 2

Identifies **Top 5 cities per category**.

Steps:

- Mapper reorganizes data by category.

- Reducer sorts cities and selects the top 5.

```

hduser@Mtech:~/ecommerce$ hdfs dfs -ls /ecommerce
2026-03-08 17:21:09,821 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r-- 1 hduser supergroup 437 2026-03-08 17:21 /ecommerce/ecommerce_transaction.csv
hduser@Mtech:~/ecommerce$ nano RevenueMapper.java
hduser@Mtech:~/ecommerce$ nano AverageReducer.java
hduser@Mtech:~/ecommerce$ nano JobIDriver.java
hduser@Mtech:~/ecommerce$ nano CategoryMapper.java
hduser@Mtech:~/ecommerce$ nano TopCitiesReducer.java
hduser@Mtech:~/ecommerce$ nano Job2Driver.java
hduser@Mtech:~/ecommerce$ javac -classpath `hadoop classpath` *.java
hduser@Mtech:~/ecommerce$ ls
AverageReducer.class  JobIDriver.class  RevenueMapper.java
AverageReducer.java   JobIDriver.java   TopCitiesReducer.class
CategoryMapper.class  Job2Driver.class  TopCitiesReducer.java
CategoryMapper.java   Job2Driver.java
ecommerce_transaction.csv RevenueMapper.class
hduser@Mtech:~/ecommerce$ jar cf ecommerce.jar *.class
hduser@Mtech:~/ecommerce$ ls
AverageReducer.class  ecommerce_transaction.csv  RevenueMapper.class
AverageReducer.java   JobIDriver.class          RevenueMapper.java
CategoryMapper.class  JobIDriver.java          TopCitiesReducer.class
CategoryMapper.java   Job2Driver.class          TopCitiesReducer.java
ecommerce.jar         Job2Driver.java
hduser@Mtech:~/ecommerce$ hadoop jar ecommerce.jar JobIDriver /ecommerce /output1
2026-03-08 17:24:13,109 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-08 17:24:15,087 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-03-08 17:24:15,272 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-03-08 17:24:15,275 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-03-08 17:24:15,806 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-03-08 17:24:16,185 INFO input.FileInputFormat: Total input files to process : 1
2026-03-08 17:24:16,345 INFO mapreduce.JobSubmitter: number of splits:1
2026-03-08 17:24:17,066 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local2010926837_0001
2026-03-08 17:24:17,066 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-03-08 17:24:19,355 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-03-08 17:24:19,370 INFO mapreduce.Job: Running job: job_local2010926837_0001
2026-03-08 17:24:19,389 INFO mapred.LocalJobRunner: OutputCommitter set in config null

```

5. Task 1 – Revenue Extraction Using Mapper

The Mapper reads the input dataset from HDFS and extracts the required fields.

Fields extracted:

- Category
- City
- Quantity
- Price

Revenue calculation:

$$\text{Revenue} = \text{Quantity} \times \text{Price}$$

The Mapper outputs key-value pairs in the following format:

Key → Category|City

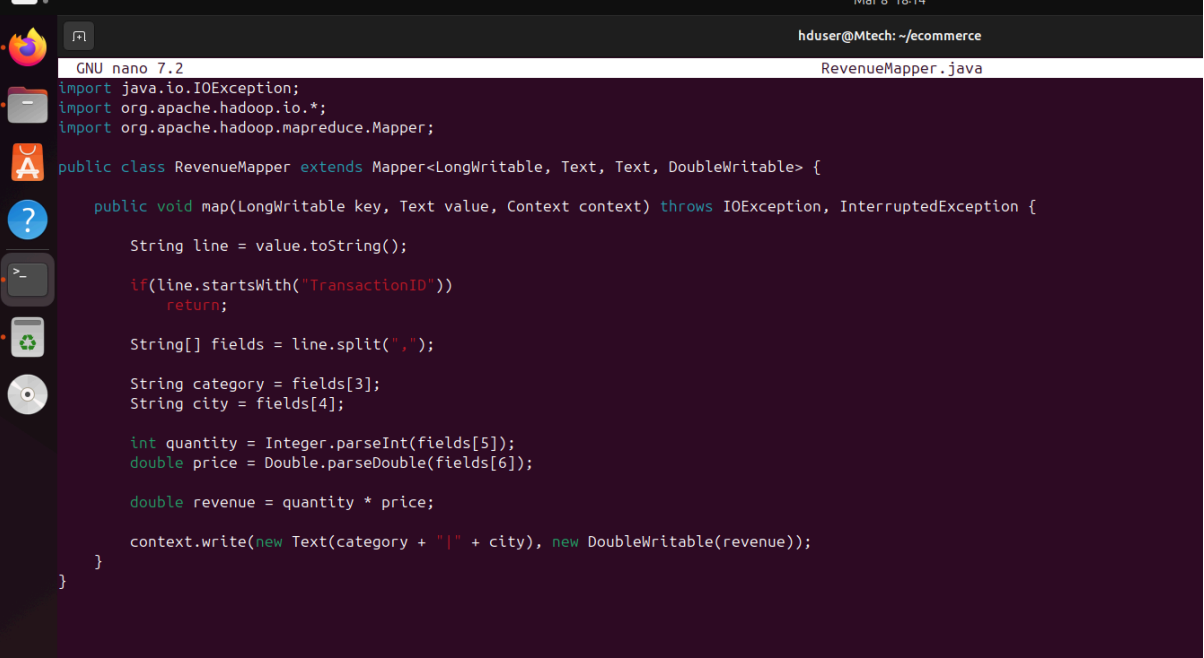
Value → Revenue

Example Mapper Output:

Electronics|Mumbai → 25000

Electronics|Chennai → 36000
Fashion|Delhi → 4500

This step converts raw transaction data into revenue records for further aggregation.



```
GNU nano 7.2 RevenueMapper.java
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class RevenueMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {

    public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {

        String line = value.toString();

        if(line.startsWith("TransactionID"))
            return;

        String[] fields = line.split(",");

        String category = fields[3];
        String city = fields[4];

        int quantity = Integer.parseInt(fields[5]);
        double price = Double.parseDouble(fields[6]);

        double revenue = quantity * price;

        context.write(new Text(category + "|" + city), new DoubleWritable(revenue));
    }
}
```

6. Task 2 – Average Revenue Calculation in Reducer

The Reducer receives all revenue values for each (Category, City) pair.

Example input to reducer:

Electronics|Mumbai → 25000
Electronics|Mumbai → 30000
Electronics|Mumbai → 20000

Reducer operations:

1. Calculate total revenue.
2. Count number of transactions.
3. Compute average revenue.

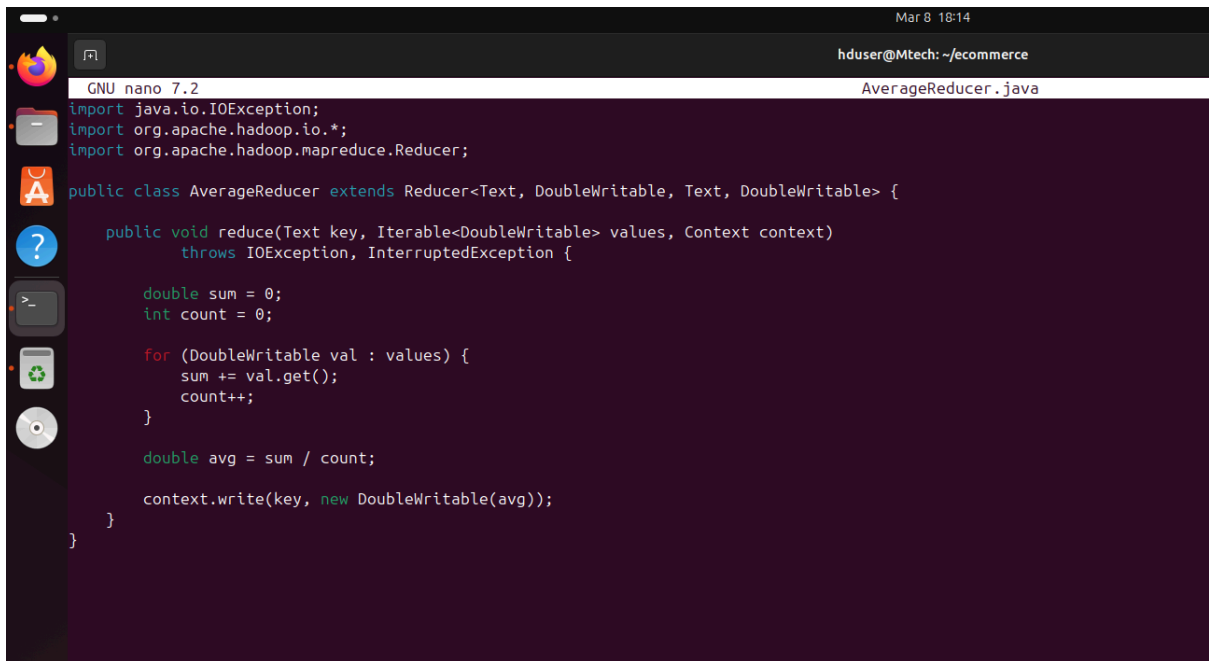
Formula:

Average Revenue = Total Revenue / Number of Transactions

Example reducer output:

Electronics|Mumbai → 25000
Electronics|Chennai → 36000
Electronics|Hyderabad → 32000
Fashion|Delhi → 4500
Fashion|Kolkata → 8800
Grocery|Bangalore → 1000

This output becomes the input for the second MapReduce job.

A screenshot of a terminal window with a dark background. The title bar at the top shows 'Mar 8 18:14' and 'hduser@Mtech: ~/ecommerce'. The terminal is running the GNU nano 7.2 text editor, editing a file named 'AverageReducer.java'. The code is written in Java and implements the Reducer interface. It imports java.io.IOException, org.apache.hadoop.io.*, and org.apache.hadoop.mapreduce.Reducer. The class 'AverageReducer' extends 'Reducer<Text, DoubleWritable, Text, DoubleWritable>'. The 'reduce' method takes a 'Text' key, an 'Iterable<DoubleWritable>' of values, and a 'Context' object. It calculates the average of the values by summing them and dividing by the count. The result is written to the context as a 'DoubleWritable'.

```
GNU nano 7.2 AverageReducer.java
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class AverageReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {

        double sum = 0;
        int count = 0;

        for (DoubleWritable val : values) {
            sum += val.get();
            count++;
        }

        double avg = sum / count;

        context.write(key, new DoubleWritable(avg));
    }
}
```

7. Task 3 – Category-wise Grouping for Ranking

The second Mapper reorganizes the output from Job 1.

It separates Category and City from the key.

Input:

Electronics|Mumbai 25000

Output:

Electronics → Mumbai:25000

Now all cities belonging to the same category are grouped together.

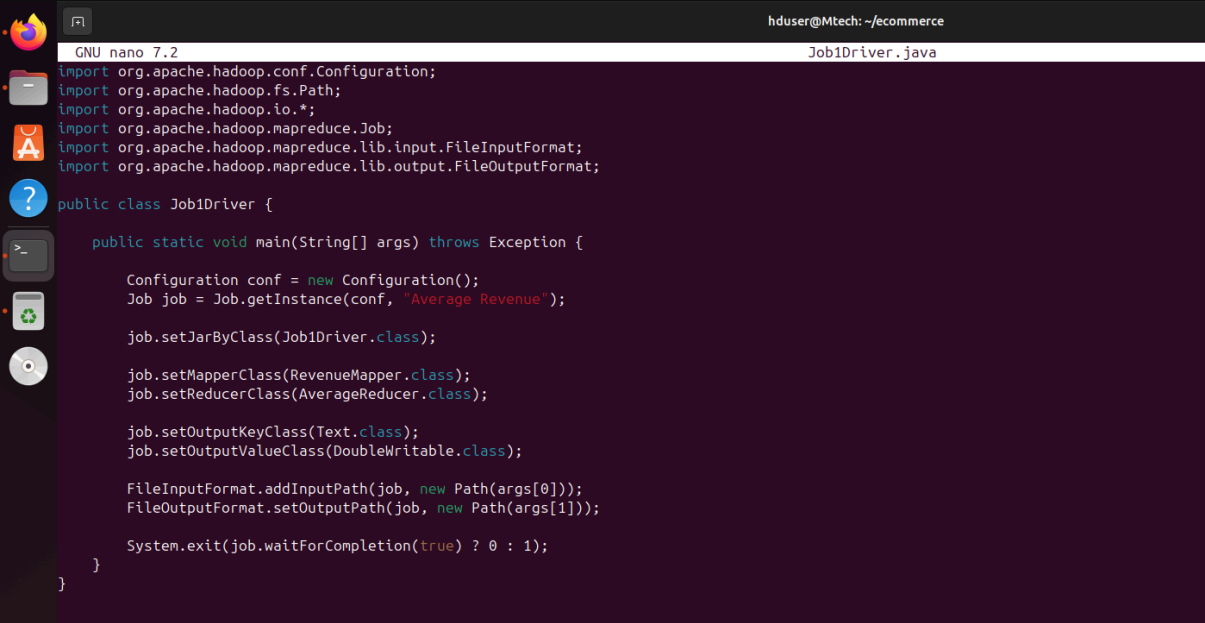
Example grouped data:

Electronics → Mumbai:25000

Electronics → Chennai:36000

Electronics → Hyderabad:32000

This prepares the dataset for ranking cities within each category.



```
GNU nano 7.2 Job1Driver.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Job1Driver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Average Revenue");

        job.setJarByClass(Job1Driver.class);

        job.setMapperClass(RevenueMapper.class);
        job.setReducerClass(AverageReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }
}
```

8. Task 4 – Top 5 Cities per Category

The Reducer receives all city revenue values belonging to the same category.

Steps performed:

1. Store all values in a list.
2. Sort the list in descending order based on average revenue.
3. Select the top 5 cities.
4. Output results.

Example:

Electronics

Chennai:36000

Hyderabad:32000

Mumbai:25000

This step identifies the cities that generate the highest revenue for each category.

```
Mar 8 18:16
hduser@Mtech: ~/ecommerce
CategoryMapper.java

GNU nano 7.2
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;

public class CategoryMapper extends Mapper<LongWritable, Text, Text, Text> {

    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String[] parts = value.toString().split("\t");

        String[] keyParts = parts[0].split("\\|");

        String category = keyParts[0];
        String city = keyParts[1];

        context.write(new Text(category), new Text(city + ":" + parts[1]));

    }
}
```

```
Mar 8 18:17
hduser@Mtech: ~/ecommerce
TopCitiesReducer.java

GNU nano 7.2
import java.io.IOException;
import java.util.*;

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;

public class TopCitiesReducer extends Reducer<Text, Text, Text, Text> {

    public void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        List<String> list = new ArrayList<>();

        for (Text val : values)
            list.add(val.toString());

        Collections.sort(list, (a,b) -> {

            double r1 = Double.parseDouble(a.split(":")[1]);
            double r2 = Double.parseDouble(b.split(":")[1]);

            return Double.compare(r2,r1);
        });

        int count = 0;

        for(String s:list){

            if(count==5)
                break;

            context.write(key,new Text(s));
            count++;
        }
    }
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

```
GNU nano 7.2 Job2Driver.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Job2Driver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Top Cities");

        job.setJarByClass(Job2Driver.class);

        job.setMapperClass(CategoryMapper.class);
        job.setReducerClass(TopCitiesReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true)?0:1);

    }
}
```

9. Intermediate Output (Job 1)

Electronics|Mumbai 25000
Electronics|Chennai 36000
Electronics|Hyderabad 32000
Fashion|Delhi 4500
Fashion|Kolkata 8800
Grocery|Bangalore 1000

```
File Output Format Counters
      Bytes Written=154
hduser@Mtech:~/ecommerce$ hdfs dfs -cat /output1/part-r-00000
2026-03-08 17:24:39,517 WARN util.NativeCodeLoader: Unable to load na
Electronics|Chennai 36000.0
Electronics|Hyderabad 32000.0
Electronics|Mumbai 25000.0
Fashion|Delhi 4500.0
Fashion|Kolkata 8800.0
Grocery|Bangalore 1000.0
hduser@Mtech:~/ecommerce$ hadoop jar ecommerce.jar Job2Driver /output
```

10. Final Output (Top 5 Cities)

Electronics Chennai:36000
Electronics Hyderabad:32000
Electronics Mumbai:25000

Fashion Kolkata:8800
Fashion Delhi:4500

Grocery Bangalore:1000

```
WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=154
File Output Format Counters
  Bytes Written=154
hduser@Mtech:~/ecommerce$ hdfs dfs -cat /finaloutput/part-r-00000
2026-03-08 17:26:20,500 WARN util.NativeCodeLoader: Unable to load native-hadoop library f
Electronics Chennai:36000.0
Electronics Hyderabad:32000.0
Electronics Mumbai:25000.0
Fashion Kolkata:8800.0
Fashion Delhi:4500.0
Grocery Bangalore:1000.0
```

11. Preprocessing Steps

Before processing the dataset, several preprocessing steps were implemented:

1. The header row of the CSV file is skipped in the Mapper to avoid processing column names.
2. The dataset is parsed using comma delimiters.
3. Invalid records are ignored to prevent runtime errors.
4. Numeric fields such as quantity and price are converted into integer and double values.
5. Revenue is computed for each transaction before being emitted.

These preprocessing steps ensure the data is clean and suitable for MapReduce processing.

12. Why Combiner Was Not Used

A Combiner performs partial aggregation at the Mapper level to reduce network traffic during the shuffle phase.

However, in this activity the program calculates **average revenue**, which requires both:

- total revenue
- number of transactions

If a Combiner calculates partial averages, the final result may become incorrect because averages cannot be directly aggregated.

For example:

Average of (10,20,30) $\neq$ average of (avg(10,20),30)

Therefore, using a Combiner without maintaining both sum and count may produce incorrect results. For this reason, the Combiner was not used in this implementation.

13. Performance Comparison (With vs Without Combiner)

Metric	Without Combiner	With Combiner
Network traffic	Higher	Lower
Mapper output size	Larger	Smaller
Accuracy	Correct	Risky for averages
Reducer workload	Higher	Lower

Since average requires complete aggregation logic, the safer implementation is **without Combiner**.

14. Challenges Faced

Several challenges were encountered while implementing this activity.

The first challenge was parsing the CSV dataset correctly in the Mapper and ensuring that the header row was skipped. Incorrect parsing could lead to runtime errors or invalid output.

The second challenge was calculating the average revenue accurately. It was necessary to track both the total revenue and the count of transactions for each category-city pair.

Another challenge was implementing the ranking logic. The reducer had to store city revenue values in a list, sort them in descending order, and extract only the top five entries.

Additionally, the multi-stage MapReduce workflow required careful management of intermediate outputs between Job 1 and Job 2.

Despite these challenges, the system successfully processed the dataset and produced the correct ranking results.

15. Learning Outcomes

After completing this activity, the following concepts were understood:

- Implementation of MapReduce aggregation
- Distributed processing of large datasets
- Understanding shuffle and reducer workload
- Calculation of distributed averages
- Category-based grouping and ranking
- Multi-stage MapReduce job execution

This activity demonstrates how Hadoop MapReduce can be used for scalable big data analytics in real-world applications such as e-commerce revenue analysis.

16. Files Submitted

The following files are included in the submission:

- Java source code for MapReduce jobs

- Sample dataset (ecommerce_transaction.csv)
 - Execution screenshots
 - Intermediate output
 - Final output file
 - Written report document
-